

UNIVERSIDAD CONTINENTAL

FACULTAD DE INGENIERÍA

**ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE
SISTEMAS E INFORMÁTICA**



PROYECTO

**“Análisis y Diseño de una Plataforma SaaS Multiempresa para la
Gestión del Reclutamiento, Evaluación y Selección de Personal –
Caso de estudio: Colegio Andino de Huancayo”**

PRESENTADO POR:

APELLIDOS Y NOMBRES	CÓDIGO
Coronacion Meza Fredy	Pendiente
Peña Arroyo Anthony	Pendiente
Vila Meza Luis Antonio	Pendiente

ASESOR:

DR. MAGLIONI ARANA CAPARACHIN

HUANCAYO – PERÚ

2026

CONTENIDO GENERAL

PORTADA	i
CONTENIDO GENERAL	ii
CAPÍTULO 1	6
INFORMACIÓN GENERAL DEL PROYECTO	6
1.1. Resumen Ejecutivo	6
1.2. Introducción	6
CAPÍTULO 2	7
CONTEXTO ORGANIZACIONAL Y ANÁLISIS DEL PROBLEMA	7
2.1. Contexto de la Organización	7
2.2. Identificación del Problema	7
CAPÍTULO 3	8
ANÁLISIS DE PROCESOS DE NEGOCIO	8
3.1. Descripción del Proceso Actual	8
3.2. Modelado del Proceso Actual (AS-IS)	8
3.3. Problemas del Proceso Actual	9
3.4. Modelado del Proceso Propuesto (TO-BE)	9
CAPÍTULO 4	10
ANÁLISIS DE REQUERIMIENTOS DEL SISTEMA	10
4.1. Identificación de Actores del Sistema	10
4.2. Requerimientos Funcionales	10
4.3. Requerimientos no Funcionales	11
4.4. Casos de Uso del Sistema	12
CAPÍTULO 5	12
PLANIFICACIÓN DEL PROYECTO Y PLAN DE CALIDAD	12
5.1. Alcance del Proyecto	12
5.2. Herramientas Tecnológicas del Proyecto	13
5.3. Normas y Estándares de Calidad	13
5.4. Plan de Pruebas del Proyecto	13

5.5. Lineamientos de Seguridad Informática	13
CAPÍTULO 6.....	14
DISEÑO DEL SISTEMA.....	14
6.1. Arquitectura Conceptual del Sistema	14
6.2. Modelo UML del Sistema	15
6.3. Diseño de Interfaces de Usuario	15
6.4. Diseño de Base de Datos	16
CAPÍTULO 7.....	17
ARQUITECTURA TECNOLÓGICA DEL SISTEMA.....	17
7.1. Tecnologías del Frontend	17
7.2. Tecnologías del Backend	17
7.3. Base de Datos del Sistema	17
7.4. Infraestructura de Desarrollo	17
CAPÍTULO 8.....	17
DESARROLLO DEL SISTEMA.....	17
8.1. Iteración 1: Configuración Inicial del Proyecto	18
8.2. Iteración 2: Desarrollo de Funcionalidades Básicas	18
8.3. Iteración 3: Implementación de Módulos Funcionales	18
8.4. Iteraciones Posteriores	18
CAPÍTULO 9.....	18
CONTROL DE VERSIONES Y GESTIÓN DEL REPOSITORIO	18
9.1. Repositorio del Proyecto	18
9.2. Estrategia de Control de Versiones	19
9.3. Gestión de Ramas del Proyecto	19
9.4. Registro de Commits Relevantes	19
CAPÍTULO 10.....	19
DOCKERIZACIÓN Y DESPLIEGUE DE MÓDULOS.....	19
10.1. Introducción a Docker en el Proyecto	19
10.2. Dockerización del Backend	19

10.3.	Dockerización del Frontend	20
10.4.	Orquestación con Docker Compose	20
CAPÍTULO 11.....		20
ESTRATEGIA DE PRUEBAS DE SOFTWARE.....		20
11.1.	Enfoque de Pruebas del Proyecto	21
11.2.	Niveles de Pruebas Aplicados	21
11.3.	Tipos de Pruebas Ejecutadas	21
11.4.	Plan de Ejecución de Pruebas	21
CAPÍTULO 12.....		21
AUTOMATIZACIÓN DE PRUEBAS		21
12.1.	Herramientas de Automatización	22
12.2.	Configuración del Entorno de Pruebas	22
12.3.	Scripts de Pruebas Automatizadas	22
12.4.	Ejecución Automática de Pruebas	22
CAPÍTULO 13.....		22
MÉTRICAS DE CALIDAD		22
13.1.	Ejecución de Casos de Pruebas	22
13.2.	Registro de Defectos	23
13.3.	Métricas de Calidad del Software	23
13.4.	Evaluación de Calidad basada en Estándares	23
CAPÍTULO 14.....		23
IMPLEMENTACIÓN Y MONITOREO.....		23
14.1.	Preparación del Entorno de Implementación	23
14.2.	Implementación del Sistema	24
14.3.	Verificación de Funcionamiento	24
14.4.	Monitoreo del Sistema	24
CONCLUSIONES		24
RECOMENDACIONES		24
REFERENCIAS		25

ANEXOS.....	25
-------------	----

CAPÍTULO 1

INFORMACIÓN GENERAL DEL PROYECTO

Este capítulo presenta la información básica del proyecto y permite comprender rápidamente el propósito del sistema desarrollado, el contexto académico del trabajo y el alcance general del proyecto.

1.1. Resumen Ejecutivo

Estados de la información usados en todo el informe: HECHO VERIFICADO (comprobado en el repositorio), AS-IS PRELIMINAR (reconstrucción del equipo, sin validación institucional), TO-BE PROPUESTO (proceso diseñado, no adoptado por la institución), SOFTWARE IMPLEMENTADO (comportamiento de la plataforma verificado con pruebas) y EXPERIMENTAL / PROPUESTO (RF-29, fuera de la línea base).

Problema (AS-IS PRELIMINAR). El análisis del proceso de reclutamiento, evaluación y selección del Colegio Andino de Huancayo (caso de estudio académico) identificó cinco problemas: información distribuida (P1), seguimiento manual (P2), evaluaciones heterogéneas (P3), comunicación manual con los postulantes (P4) e indicadores limitados para sustentar la decisión (P5). No hay validación institucional de este análisis.

Solución (SOFTWARE IMPLEMENTADO). Plataforma web SaaS multiempresa que cubre el proceso completo en 27 requerimientos funcionales (RF-01 a RF-27): requerimiento de personal y su aprobación, vacante con perfil y criterios ponderados, postulación con CV, preselección y etapas, evaluaciones y entrevistas, ranking explicable, decisión final, selección, cierre, notificaciones y auditoría. El ranking calcula, ordena y compara; **la decisión final la registra una persona** (Aprobador / Dirección, RF-23).

Tecnologías. Laravel 13 (PHP 8.4), React 19 con TypeScript e Inertia 3, Tailwind 4, PostgreSQL 17, Redis 7 y Docker Compose. La versión v1.1 agrega un servicio experimental (Python, scikit-learn y FastAPI) que estima el riesgo de demora del **proceso** (RF-29, EXPERIMENTAL): no evalúa, ordena ni selecciona personas.

Enfoque de calidad. Desarrollo guiado por pruebas (TDD), pruebas automatizadas en cuatro suites, integración continua en GitHub Actions, registro de defectos con prueba de regresión e ISO/IEC 25010 como marco de referencia (sin certificación).

Resultados (HECHO VERIFICADO, ejecución QA del 30/09/2026 sobre el commit bc44303). PHPUnit 411 pruebas aprobadas y 8 omitidas, sin fallos; Cypress 85/85; Vitest 42/42; pytest 533/533; TypeScript y build sin errores. Los 27 RF tienen al menos un caso de prueba automatizado aprobado. No se midió cobertura de código ni rendimiento, y no hay beneficios medidos en la institución.

1.2. Introducción

Contexto. La gestión del talento en una institución educativa exige procesos trazables y comparables. El proyecto toma como caso de estudio académico al Colegio Andino de Huancayo y diseña una plataforma SaaS: una sola aplicación que atiende a varias organizaciones con los datos de cada una aislados (`organization_id`).

Importancia de las pruebas. El proyecto pertenece al curso Pruebas y Calidad de Software. Cada regla de negocio se escribió primero como una prueba que falla (TDD); la regresión

automática protege cada cambio; la integración continua repite build, TypeScript y PHPUnit en cada push; y cada defecto encontrado quedó registrado con su prueba de regresión.

Propósito. Analizar, diseñar, implementar y verificar una plataforma que atienda los problemas identificados, con la decisión humana de selección y la evidencia verificable de calidad como requisitos centrales.

Estructura. Los capítulos 2 y 3 describen el contexto y el proceso (AS-IS y TO-BE); el 4, los requerimientos; el 5, la planificación y el plan de calidad; el 6 y el 7, el diseño y la arquitectura; el 8 al 10, el desarrollo, el control de versiones y la dockerización; el 11 al 13, la estrategia, la automatización y las métricas de pruebas; y el 14, la implementación y el monitoreo. Siguen las conclusiones, las recomendaciones, las referencias y los anexos.

CAPÍTULO 2

CONTEXTO ORGANIZACIONAL Y ANÁLISIS DEL PROBLEMA

Este capítulo describe el entorno donde se identifica el problema y explica por qué es necesario desarrollar una solución informática.

2.1. Contexto de la Organización

El Colegio Andino de Huancayo es el **caso de estudio académico** del proyecto. El repositorio no contiene documentación institucional verificada: no hay estadísticas de contratación, número de postulantes, organigrama ni herramientas actuales confirmadas. El contexto se describe como AS-IS PRELIMINAR, pendiente de validación con RR. HH. y la Administración del colegio.

- **Actividades principales (AS-IS PRELIMINAR):** 1) Detectar la necesidad de personal y comunicarla; 2) Revisar y aprobar la necesidad; 3) Definir el perfil del puesto y difundir la convocatoria; 4) Recibir las postulaciones y los CV; 5) Revisar y preseleccionar candidatos; 6) Coordinar evaluaciones y entrevistas; 7) Consolidar resultados y comparar candidatos; 8) Decidir y comunicar el resultado.
- **Área donde se presenta el problema:** reclutamiento, evaluación y selección de docentes y personal, con participación del área solicitante, RR. HH., la Dirección, los evaluadores y los postulantes.
- **Herramientas actuales:** no identificadas. El AS-IS preliminar no identifica ninguna herramienta informática del Colegio, y no se afirma que el proceso sea en papel, por correo ni con otra herramienta: queda pendiente de validación.

Fuentes: Formato 02 (análisis del proceso) en docs/academico/practica-02 y capítulo 2 de la documentación v1.0.

2.2. Identificación del Problema

Problemas del proceso actual (AS-IS PRELIMINAR, Formato 04). Ninguno se midió: son hallazgos del análisis del equipo pendientes de validación.

ID	Problema	Descripción	Causa	Impacto (consecuencia operativa)	Prioridad
P1	Información distribuida	Los datos del proceso (requerimiento, perfil del puesto, CV, evaluaciones y decisiones) no están centralizados en un único registro por convocatoria.	Ausencia de un registro único por convocatoria	Dificulta reconstruir el expediente y el estado integral de una postulación; riesgo de pérdida o duplicación de información y menor trazabilidad.	Alta

ID	Problema	Descripción	Causa	Impacto (consecuencia operativa)	Prioridad
P2	Seguimiento manual	El avance de cada requerimiento y postulación se controla sin estados ni historial sistematizados.	Estados del proceso no definidos ni registrados	Poca visibilidad del estado del proceso y de quién hizo cada cambio; más esfuerzo para controlar etapas y responsables.	Alta
P3	Evaluaciones heterogéneas	Los candidatos no se evalúan con criterios, ponderaciones y rangos definidos de antemano y comunes a toda la convocatoria.	Criterios de evaluación no definidos antes de evaluar	Comparaciones poco objetivas y difíciles de justificar; menor comparabilidad de resultados.	Alta
P4	Comunicación manual	Los avisos a postulantes y participantes (recepción, cambios de etapa, convocatorias y resultado) dependen de gestiones manuales.	Avisos que dependen de una gestión manual en cada evento	Demoras u omisiones en la comunicación; es difícil comprobar qué se comunicó y cuándo.	Media
P5	Indicadores limitados	No se dispone de información consolidada para comparar candidatos ni de trazabilidad de las acciones críticas.	Datos no consolidados y sin registro de acciones críticas	Decisiones con menos sustento, baja capacidad de auditoría y dificultad para medir demoras y resultados del proceso.	Media

Impacto en la eficiencia del proceso: la falta de un registro único, de estados sistematizados y de criterios comunes dificulta reconstruir cada convocatoria, justificar la decisión y comunicar a tiempo a los postulantes. No se cuantificó ese impacto (no hay línea base medida).

CAPÍTULO 3

ANÁLISIS DE PROCESOS DE NEGOCIO

Este capítulo analiza cómo funcionan los procesos organizacionales antes de implementar el sistema. Se utilizan técnicas de **Business Process Management (BPM)** para comprender el flujo de trabajo actual.

3.1. Descripción del Proceso Actual

El proceso actual (AS-IS PRELIMINAR) se modeló en el Formato 02 con 14 actividades (AS-01 a AS-14) agrupadas en 8 macroactividades:

#	Macroactividad	Responsable	Problemas
1	Detectar la necesidad de personal y comunicarla	Área solicitante	P1, P2
2	Revisar y aprobar la necesidad	RR. HH. / Dirección	P2
3	Definir el perfil del puesto y difundir la convocatoria	RR. HH.	P1
4	Recibir las postulaciones y los CV	RR. HH.	P1, P4
5	Revisar y preseleccionar candidatos	RR. HH.	P2
6	Coordinar evaluaciones y entrevistas	RR. HH. / Evaluadores	P3, P4
7	Consolidar resultados y comparar candidatos	RR. HH.	P3, P5
8	Decidir y comunicar el resultado	Dirección / RR. HH.	P4, P5

3.2. Modelado del Proceso Actual (AS-IS)

El diagrama BPMN AS-IS se formalizó en PowerDesigner (Formato 03 y fase F29): pools del Colegio y del Postulante, carriles por actor y mensajes entre pools. Se reproduce en el **Anexo A**.

ID	Actividad	Descripción
AS-01	Identificar la necesidad de personal	El área detecta que requiere cubrir un puesto (vacante, reemplazo o nueva plaza).
AS-02	Comunicar la necesidad a RR. HH.	El área transmite la necesidad a RR. HH. El canal y el formato no están verificados.
AS-03	Revisar la necesidad	RR. HH. revisa la necesidad comunicada antes de elevarla a Dirección.

ID	Actividad	Descripción
AS-04	Aprobar o no aprobar la necesidad	Dirección decide si la necesidad procede. Si no procede, el proceso termina para esa necesidad.
AS-05	Definir el perfil del puesto	RR. HH. define el perfil del puesto a cubrir.
AS-06	Difundir la convocatoria	RR. HH. da a conocer la convocatoria. Los medios de difusión no están verificados.
AS-07	Presentar la postulación y el CV	La persona interesada presenta su postulación y su CV.
AS-08	Recibir y reunir postulaciones y CV	RR. HH. recibe y reúne las postulaciones de la convocatoria.
AS-09	Revisar el CV y preseleccionar al candidato	RR. HH. revisa el CV de cada candidato y decide si continúa (dentro de SP-01, una vez por candidato). No está verificado si se informa a quien no continúa.
AS-10	Coordinar evaluaciones y entrevistas	RR. HH. coordina fechas y participantes, y cita a los candidatos preseleccionados.
AS-11	Realizar evaluaciones y entrevistas	Los evaluadores evalúan y entrevistan a los candidatos y entregan sus resultados. No hay constancia de criterios, ponderaciones ni rangos comunes definidos de antemano.
AS-12	Consolidar resultados y comparar candidatos	RR. HH. reúne los resultados de cada candidato y los compara.
AS-13	Decidir el candidato seleccionado	Dirección decide a quién seleccionar a partir de la información consolidada.
AS-14	Comunicar el resultado	RR. HH. comunica el resultado a los postulantes. El canal y el alcance de la comunicación no están verificados.

3.3. Problemas del Proceso Actual

Relación entre los problemas y las actividades del AS-IS en las que aparecen (Formato 04):

Problema	Actividades AS-IS	Validación pendiente con
P1 Información distribuida	AS-02, AS-05, AS-06, AS-08	RR. HH.
P2 Seguimiento manual	AS-02, AS-03, AS-04, AS-09	RR. HH.
P3 Evaluaciones heterogéneas	AS-11, AS-12	RR. HH. / Dirección
P4 Comunicación manual	AS-07, AS-10, AS-14	RR. HH.
P5 Indicadores limitados	AS-12, AS-13	Dirección

3.4. Modelado del Proceso Propuesto (TO-BE)

El proceso propuesto (TO-BE PROPUESTO, Formato 05) atiende cada problema con una solución y un objetivo de mejora. El BPMN TO-BE formal de PowerDesigner está en el **Anexo B**. Su adopción por la institución no está validada.

Problema	Solución	Descripción	RF
P1	S-01 Centralización de la información	Registro único por convocatoria en la plataforma multiempresa: requerimiento, vacante con perfil y criterios, perfil del postulante con CV privado y expediente de postulación.	RF-01, RF-05, RF-06, RF-09, RF-10, RF-12
P2	S-02 Control del avance y responsables identificados	Estados e historial sistematizados del requerimiento y de la postulación (quién, cuándo y de qué estado a cuál), con transiciones controladas.	RF-02, RF-03, RF-13, RF-14, RF-24, RF-25
P3	S-03 Evaluación uniforme y comparable	Criterios con ponderación y rango definidos antes de publicar; puntajes validados; ranking ponderado explicable y comparación común.	RF-05, RF-16, RF-17, RF-18, RF-19, RF-20, RF-21, RF-22
P4	S-04 Comunicación oportuna y con constancia	Notificaciones automáticas en cada evento clave: rechazo, recepción, cambio de etapa, convocatoria y resultado. No incluyen datos confidenciales.	RF-04, RF-11, RF-15, RF-17, RF-26
P5	S-05 Sustento y trazabilidad de la decisión (parcial)	Comparación explicable como apoyo a la decisión humana y auditoría de solo inserción consultable por Dirección. Límite: los indicadores de gestión del proceso (tiempos, embudo) no se implementan; RF-28 es un candidato no implementado.	RF-21, RF-22, RF-23, RF-27

Objetivo de mejora	Descripción	Problema
OM-01	Centralizar la información de cada convocatoria en un único registro por organización.	P1
OM-02	Controlar el avance con estados, transiciones válidas e historial con autor y fecha.	P2
OM-03	Evaluar con criterios, ponderaciones y rangos comunes definidos antes de publicar.	P3
OM-04	Eliminar la dependencia de avisos manuales en los eventos clave del proceso.	P4
OM-05	Dar sustento y trazabilidad a la decisión final, que sigue siendo humana.	P5

CAPÍTULO 4

ANÁLISIS DE REQUERIMIENTOS DEL SISTEMA

Este capítulo documenta los requerimientos del sistema que será desarrollado. El análisis de requerimientos permite definir **qué debe hacer el sistema antes de comenzar su implementación.**

4.1. Identificación de Actores del Sistema

ID	Actor	Rol en el sistema	Tipo
ACT-01	Área solicitante	solicitante	Interno
ACT-02	Recursos Humanos (RR. HH.)	rrhh	Interno
ACT-03	Aprobador / Dirección	aprobador	Interno
ACT-04	Postulante	postulante	Externo
ACT-05	Evaluador	evaluador	Interno

Además actúa el **Sistema** en las notificaciones automáticas (RF-04, RF-11, RF-15, RF-17 y RF-26). No hay rol de superadministración (fuera de alcance, OUT-03).

4.2. Requerimientos Funcionales

Línea base de 27 requerimientos funcionales (Formato 06), todos SOFTWARE IMPLEMENTADO y con prueba automatizada. RF-28 es candidato no implementado y RF-29 es EXPERIMENTAL / PROPUESTO: ninguno de los dos forma parte de la línea base.

RF	Requerimiento	Descripción	Actor
RF-01	Registrar requerimiento de personal	El sistema debe permitir al área solicitante registrar un requerimiento de personal.	Área solicitante
RF-02	Validar y corregir requerimiento	El sistema debe permitir enviar el requerimiento a RR. HH., que lo valida u observa, y al área corregir y reenviar lo observado.	Área solicitante (enviar, corregir y reenviar) · RR. HH. (validar u observar)
RF-03	Registrar aprobación o rechazo del requerimiento	El sistema debe permitir al Aprobador / Dirección aprobar o rechazar un requerimiento validado, con motivo obligatorio si lo rechaza.	Aprobador / Dirección
RF-04	Notificar rechazo del requerimiento	El sistema debe notificar automáticamente al área solicitante el rechazo de su requerimiento, con el motivo.	Sistema (destinatario: área solicitante)
RF-05	Registrar perfil y criterios del puesto	El sistema debe permitir a RR. HH. crear una vacante desde un requerimiento aprobado y registrar su perfil y sus criterios ponderados.	RR. HH.
RF-06	Configurar y validar vacante	El sistema debe permitir configurar la convocatoria y mostrar una lista de validación previa a la publicación.	RR. HH. (configura) · Sistema (valida)
RF-07	Publicar vacante	El sistema debe publicar en el portal público de empleos las vacantes que cumplan la validación.	RR. HH.
RF-08	Gestionar cuenta y acceso del postulante	El sistema debe permitir al postulante crear su cuenta e iniciar sesión.	Postulante
RF-09	Gestionar perfil y CV del postulante	El sistema debe permitir al postulante completar su perfil y cargar su CV en PDF de forma privada.	Postulante
RF-10	Registrar postulación	El sistema debe registrar una única postulación por postulante y vacante publicada y vigente.	Postulante
RF-11	Confirmar postulación al postulante	El sistema debe confirmar cada postulación con un código de seguimiento y un aviso de recepción.	Sistema (destinatario: postulante)
RF-12	Consultar y revisar postulaciones	El sistema debe listar las postulaciones de cada vacante y mostrar el expediente con datos, CV e historial.	RR. HH. (también consulta el Aprobador / Dirección)
RF-13	Registrar preselección o descarte	El sistema debe permitir a RR. HH. preseleccionar o descartar una postulación. El descarte lleva un motivo interno.	RR. HH.
RF-14	Gestionar cambio de etapa de la postulación	El sistema debe permitir a RR. HH. cambiar la etapa de una postulación según las transiciones permitidas, con historial.	RR. HH.
RF-15	Notificar cambio de etapa al candidato	El sistema debe avisar al postulante de cada cambio de etapa, sin revelar observaciones internas.	Sistema (destinatario: postulante)
RF-16	Programar evaluación	El sistema debe permitir a RR. HH. programar una evaluación con un evaluador de la organización.	RR. HH.

RF	Requerimiento	Descripción	Actor
RF-17	Generar convocatoria de evaluación	El sistema debe generar la convocatoria al postulante y el aviso de asignación al evaluador.	Sistema (destinatarios: postulante y evaluador)
RF-18	Programar entrevista	El sistema debe permitir a RR. HH. programar una entrevista con un evaluador asignado y su convocatoria.	RR. HH.
RF-19	Registrar entrevista y su resultado	El sistema debe permitir al evaluador asignado registrar, una sola vez, los puntajes, el resultado y las observaciones de su sesión.	Evaluador asignado
RF-20	Validar rangos y ponderaciones	El sistema debe validar que las ponderaciones y los rangos de los criterios sean coherentes y que cada puntaje esté dentro de su rango.	Sistema (lo configura RR. HH.)
RF-21	Calcular ranking configurable	El sistema debe calcular un ranking ponderado, determinista y explicable, sin seleccionar a ningún candidato.	Sistema
RF-22	Presentar comparación de candidatos	El sistema debe presentar la comparación de candidatos con criterios, promedios, aportes, total y posición.	RR. HH. - Aprobador / Dirección
RF-23	Registrar decisión final de selección	El sistema debe permitir al Aprobador / Dirección registrar la decisión final humana, con confirmación explícita y justificación.	Aprobador / Dirección
RF-24	Registrar selección del candidato	El sistema debe permitir a RR. HH. aplicar la decisión final registrando la selección del candidato decidido.	RR. HH.
RF-25	Cerrar vacante o convocatoria	El sistema debe permitir a RR. HH. cerrar la convocatoria después de registrar la selección.	RR. HH.
RF-26	Notificar resultado y cierre al postulante	El sistema debe notificar a cada postulante su propio resultado al cerrar la convocatoria.	Sistema (destinatarios: postulantes)
RF-27	Generar registro de auditoría	El sistema debe registrar de forma de solo inserción las acciones críticas y permitir al Aprobador / Dirección consultarlas.	Sistema (registro) - Aprobador / Dirección (consulta)

4.3. Requerimientos no Funcionales

Diez RNF académicos (Formato 07), clasificados con ISO/IEC 25010 como marco de referencia. Estado: 5 verificados, 3 con evidencia parcial y 2 no verificados.

RNF	Característica	Nombre	Criterio	Estado
RNF-01	Seguridad	Seguridad y control de acceso	Toda ruta del personal exige sesión y rol (middleware role y Policies). Contraseñas con hash bcrypt. Límite de 5 intentos de inicio de sesión por minuto. Las pruebas de autorización por rol pasan sin fallos.	VERIFICADO
RNF-02	Seguridad	Multitenencia y aislamiento	Toda entidad de negocio lleva organization_id y un scope global. Las Policies comparan rol y organización. Las pruebas cross-tenant pasan sin fallos.	VERIFICADO
RNF-03	Seguridad (responsabilidad)	Trazabilidad y auditoría	Cada acción crítica deja un registro. La tabla audit_logs rechaza UPDATE y DELETE mediante un trigger. Los historiales de estado registran quién, cuándo y de qué estado a cuál.	VERIFICADO
RNF-04	Seguridad (confidencialidad)	Privacidad	No se almacenan DNI, fecha de nacimiento ni atributos sensibles no requeridos. El CV se guarda en disco privado con nombre UUID y solo lo descargan los usuarios autorizados por la Policy. Las notificaciones no incluyen datos de otros candidatos.	VERIFICADO
RNF-05	Usabilidad	Usabilidad	Interfaz en español; errores por campo; estados vacíos; navegación por rol; diseño adaptable de 320 a 1440 px; contraste AA en modo claro y oscuro; foco visible y enlace de salto.	EVIDENCIA PARCIAL
RNF-06	Eficiencia de desempeño	Rendimiento	Criterio propuesto: fijar un entorno de referencia, medir una línea base de las operaciones críticas (listado de postulaciones, comparación, registro de decisión) y aprobar umbrales antes de cualquier aceptación productiva. No hay umbral aprobado.	NO VERIFICADO
RNF-07	Fiabilidad	Disponibilidad y recuperabilidad	Criterio propuesto: procedimiento de respaldo y restauración de la base probado, y objetivos de disponibilidad aprobados. No definidos.	NO VERIFICADO
RNF-08	Compatibilidad	Compatibilidad	Criterio propuesto: lista de navegadores y versiones aprobada, y pruebas en cada uno. No hay matriz formal.	EVIDENCIA PARCIAL
RNF-09	Mantenibilidad	Mantenibilidad	Monolito modular por dominio: controlador delgado → Form Request → Policy → servicio → modelo, con máquinas de estado en enums. Suites automatizadas de regresión en CI: build, tsc y PHPUnit.	EVIDENCIA PARCIAL
RNF-10	Seguridad (integridad)	Integridad de datos	Form Requests en cada operación; máquinas de estado que rechazan transiciones inválidas; validadores de ponderaciones y puntajes; restricciones CHECK, UNIQUE y FK; un solo seleccionado por vacante; los IDs no numéricos devuelven 404.	VERIFICADO

4.4. Casos de Uso del Sistema

Veinte casos de uso (Formato 08), formalizados en PowerDesigner. El diagrama está en el **Anexo C**.

CU	Caso de uso	Descripción	RF
CU-01	Registrar requerimiento de personal	El área registra la necesidad de personal.	RF-01
CU-02	Validar y corregir requerimiento	RR. HH. valida u observa; el área corrige y reenvía.	RF-02
CU-03	Aprobar o rechazar requerimiento	La Dirección decide sobre el requerimiento validado; el rechazo se notifica.	RF-03, RF-04
CU-04	Registrar perfil y criterios del puesto	RR. HH. crea la vacante con perfil y criterios ponderados.	RF-05
CU-05	Configurar y validar vacante	RR. HH. configura la convocatoria; el sistema valida.	RF-06
CU-06	Publicar vacante	RR. HH. publica la vacante válida en el portal.	RF-07
CU-07	Gestionar cuenta y acceso	El postulante crea su cuenta e inicia sesión.	RF-08
CU-08	Gestionar perfil y CV	El postulante completa su perfil y carga su CV.	RF-09
CU-09	Registrar postulación	El postulante postula y recibe la confirmación.	RF-10, RF-11
CU-10	Consultar y revisar postulaciones	Revisa el listado por vacante y el expediente.	RF-12
CU-11	Registrar preselección o descarte	RR. HH. preselecciona o descarta con motivo interno.	RF-13
CU-12	Gestionar cambio de etapa	RR. HH. cambia la etapa; el postulante recibe el aviso.	RF-14, RF-15
CU-13	Programar evaluación	RR. HH. programa la evaluación; se envía la convocatoria.	RF-16, RF-17
CU-14	Programar entrevista	RR. HH. programa la entrevista; se envía la convocatoria.	RF-18
CU-15	Registrar resultados de evaluación y entrevista	El evaluador asignado registra puntajes y resultado.	RF-19
CU-16	Validar rangos y ponderaciones	Validación incluida por la configuración, el registro de resultados y el ranking.	RF-20
CU-17	Consultar ranking y comparación	Ranking y comparación explicables. No selecciona.	RF-21, RF-22
CU-18	Registrar decisión final humana	Decisión humana del Aprobador / Dirección, con confirmación explícita y justificación. El ranking no elige.	RF-23
CU-19	Registrar selección	RR. HH. aplica la decisión registrada.	RF-24
CU-20	Cerrar convocatoria y notificar resultado	RR. HH. cierra con selección; cada postulante recibe su resultado.	RF-25, RF-26

CAPÍTULO 5

PLANIFICACIÓN DEL PROYECTO Y PLAN DE CALIDAD

Este capítulo define cómo se organizará el desarrollo del sistema antes de iniciar la implementación. Incluye la planificación del proyecto, el plan de pruebas, los estándares de calidad y las herramientas tecnológicas que se utilizarán.

5.1. Alcance del Proyecto

Alcance del Formato 09 (entregable final v1.1): ocho bloques incluidos y once exclusiones explícitas.

Incluido	Bloque
IN-01	Requerimientos
IN-02	Vacantes
IN-03	Cuenta y postulación
IN-04	Seguimiento
IN-05	Evaluación y entrevista
IN-06	Comparación y ranking
IN-07	Decisión y cierre
IN-08	Auditoría

Excluido	Descripción
OUT-01, OUT-02	Facturación SaaS, planes y suscripciones
OUT-03	Superadministración comercial

Excluido	Descripción
OUT-04, OUT-05	Selección automática por IA o ML e inferencia de personalidad o idoneidad
OUT-06, OUT-07	Banco de talentos; dashboards e indicadores avanzados
OUT-08 a OUT-10	Proveedores externos, infraestructura productiva en la nube, integraciones no especificadas
OUT-11	Uso productivo o decisorio del riesgo operacional ML (RF-29)

5.2. Herramientas Tecnológicas del Proyecto

Área	Herramienta
Backend	PHP 8.4, Laravel 13, Fortify, Inertia 3
Frontend	React 19, TypeScript, Tailwind 4, Vite
Datos	PostgreSQL 17, Redis 7
ML experimental	Python 3.12, scikit-learn, FastAPI (fuera de Compose)
Pruebas	PHPUnit, Cypress 15.3.0, Vitest, pytest
Entorno	Docker Desktop y Docker Compose
Versiones y CI	Git, GitHub y GitHub Actions
Modelado	PowerDesigner (BPMN, UML y modelo físico)
Asistencia	Claude (implementación asistida) y Codex (auditoría), bajo dirección del equipo

5.3. Normas y Estándares de Calidad

ISO/IEC 25010 se usa como **marco de referencia** para clasificar los RNF y organizar la evaluación de calidad (capítulo 13). No se declara certificación ni conformidad con la norma.

- Estándares de práctica: TDD, integración continua, revisión independiente de cada fase y registro de defectos con prueba de regresión.
- Accesibilidad: criterios WCAG 2.2 revisados en la Fase 21 (contraste AA, reflow y foco visible), sin auditoría formal externa.
- Datos: solo ficticios; ninguna información personal real.

5.4. Plan de Pruebas del Proyecto

El plan formal está en el Plan de Pruebas de Software (F29D, docs/academico/plan-pruebas/), sobre la plantilla del curso. Define alcance, estrategia en niveles, criterios de aceptación CA-01 a CA-07, suspensión y reanudación, recursos, RACI, cronograma y riesgos. No está aprobado ni firmado: la aprobación académica está pendiente.

Criterio	Descripción
CA-01	PHPUnit sin fallos; solo las 8 omitidas del starter kit
CA-02	Cypress 20 specs sin fallos ni reintentos
CA-03	Vitest y pytest sin fallos
CA-04	TypeScript y build sin errores
CA-05	CI en verde en develop
CA-06	Cada RF-01 a RF-27 con un caso automatizado aprobado
CA-07	Ningún defecto Alto abierto

5.5. Lineamientos de Seguridad Informática

- Autenticación con Fortify; contraseñas con hash; límite de intentos de inicio de sesión (RNF-01).
- Autorización en el servidor con Policies que comparan rol y organización; middleware de rol.
- Aislamiento multiempresa por organization_id con scope global y pruebas de acceso cruzado (RNF-02).

- Auditoría de solo inserción: la tabla audit_logs rechaza UPDATE y DELETE con un trigger (RNF-03).
- Privacidad: perfil mínimo del postulante y CV en almacenamiento privado (RNF-04).
- Endpoints de soporte E2E inertes fuera del entorno aislado y protegidos por token (DEF-13).
- Secretos fuera de Git (.env, .env.e2e); Postgres y Redis publicados solo en 127.0.0.1.
- Límites: sin pruebas de penetración ni análisis dinámico (OWASP ZAP); RLS de PostgreSQL no implementado.

CAPÍTULO 6

DISEÑO DEL SISTEMA

Este capítulo describe el diseño conceptual del sistema antes de iniciar su implementación. El diseño permite definir cómo se estructurará la solución tecnológica para cumplir con los requerimientos identificados.

6.1. Arquitectura Conceptual del Sistema

Monolito modular SaaS multiempresa (Formato 11, regularizado en la F11-R). Diecisiete componentes conceptuales (CMP-01 a CMP-17) y veinte relaciones (R-01 a R-20). CMP-17 (RF-29) es EXPERIMENTAL y opcional: la aplicación funciona igual sin él.

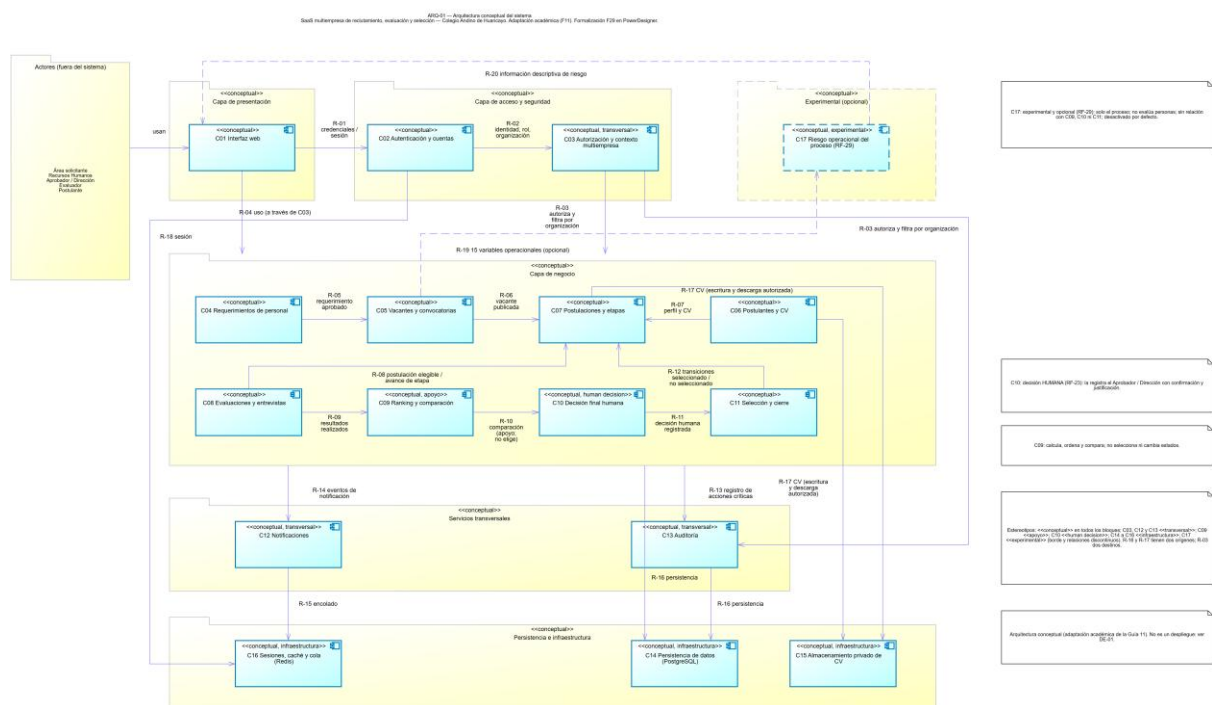
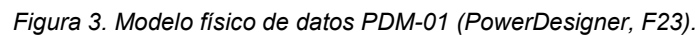


Figura 1. Arquitectura conceptual ARQ-01 (PowerDesigner, F29).

CMP	Componente	Capa
CMP-01	Interfaz web	Presentación
CMP-02	Autenticación y cuentas	Acceso y seguridad
CMP-03	Autorización y contexto multiempresa	Acceso y seguridad
CMP-04	Requerimientos de personal	Negocio
CMP-05	Vacantes y convocatorias	Negocio

- Accesibilidad verificada con pruebas automatizadas (E2E-16 a E2E-19 y componentes Vitest): contraste AA, main en todas las pantallas, enlace de salto, foco visible, reflow a 320 px.
- La tarjeta de riesgo operacional (RF-29) se muestra solo en la ficha de la vacante, rotulada como experimental, y nunca junto a candidatos ni al ranking.

Modelo físico PostgreSQL con restricciones de integridad (claves foráneas, CHECK y únicos), `organization_id` en las entidades de negocio y un trigger que convierte `audit_logs` en tabla de solo inserción. Se formalizó como modelo físico en PowerDesigner (PDM-01 y PDM-02, Fase 23).



CAPÍTULO 7

ARQUITECTURA TECNOLÓGICA DEL SISTEMA

Este capítulo describe las tecnologías utilizadas para implementar el sistema y cómo interactúan entre sí.

7.1. Tecnologías del Frontend

Tecnología	Uso
React 19 + TypeScript	Páginas y componentes
Inertia 3	Puente SPA con Laravel, sin API separada
Tailwind 4 (componentes shadcn/ui)	Estilos y sistema de diseño
Vite (vite-plus vp)	Empaquetado, pruebas de componentes y formato
Laravel Wayfinder	Rutas tipadas

7.2. Tecnologías del Backend

Tecnología	Uso
PHP 8.4 + Laravel 13	Aplicación y reglas de negocio (monolito modular)
Fortify	Autenticación
Polices y Form Requests	Autorización y validación en el servidor
Colas con Redis	Notificaciones asíncronas enviadas tras confirmar la transacción
Servicio ML (FastAPI)	RF-29 EXPERIMENTAL, llamado solo si <code>ML_SERVICE_ENABLED=true</code>

7.3. Base de Datos del Sistema

- PostgreSQL 17 con tres bases: reclutamiento (desarrollo y demostración), reclutamiento_testing (PHPUnit) y reclutamiento_e2e (Cypress). Las pruebas nunca tocan la de demostración.
- Redis 7 para sesiones, caché y colas; el entorno E2E usa sus propias bases de Redis.
- Datos de demostración ficticios generados por DemoSeeder (docs/demo-users.md).

7.4. Infraestructura de Desarrollo

Todo el entorno corre en Docker Compose con versiones fijadas; no se usan PHP ni Node del equipo host. Servicios: app, queue, postgres, redis y, en el perfil e2e, app-e2e, queue-e2e y cypress. El servicio ML corre fuera de Compose, en su entorno virtual. La integración continua usa GitHub Actions.

CAPÍTULO 8

DESARROLLO DEL SISTEMA

Este capítulo documenta el proceso de desarrollo del software mediante iteraciones o incrementos del sistema. Cada iteración representa un conjunto de funcionalidades implementadas durante el desarrollo del proyecto. Este capítulo es clave porque permite documentar **cómo se construyó realmente el sistema**.

8.1. Iteración 1: Configuración Inicial del Proyecto

- Repositorio Git con main y develop; bootstrap de Laravel (e87ef39).
- Frontend React + Inertia + TypeScript y backend Laravel desde el kit inicial.
- PostgreSQL y Redis en Docker; autenticación, organizaciones, roles y aislamiento multiempresa (feature/tenancy-roles, merge d696032).
- Defectos DEF-01 y DEF-02 (entorno Docker y variables de PHPUnit), corregidos en esta iteración.

8.2. Iteración 2: Desarrollo de Funcionalidades Básicas

- Requerimientos de personal y vacantes: RF-01 a RF-07 (merge 5ab6e22), con máquinas de estado y validación de ponderaciones.
- Primeras páginas Inertia conectadas a controladores delgados que delegan en servicios.
- Ciclos TDD registrados en docs/tdd-evidence.md.

8.3. Iteración 3: Implementación de Módulos Funcionales

- Postulantes y postulaciones: RF-08 a RF-15 (merge f20f0b6).
- Evaluaciones y entrevistas: RF-16 a RF-19 (merge 40eda7d), con checkpoint RED previo (3e66623).
- Ranking, decisión humana, selección y cierre: RF-20 a RF-25 (merge dda4bd0).
- Notificaciones y auditoría: RF-26 y RF-27 (merge cd092d1). Formularios, validaciones y operaciones CRUD con autorización por rol y organización.

8.4. Iteraciones Posteriores

Iteración	Contenido	Problemas y soluciones
Fase 8	Frontend integral y validación manual en navegador	DEF-09 a DEF-11 (zona horaria, etiquetas, iniciales)
Fases 9 y 10	Suite E2E con Cypress; Docker y entorno E2E aislado	DEF-12 y DEF-13 (fechas E2E, reset aislado)
Fases 13 a 17	Gobierno de v1.1; servicio ML experimental y su integración (RF-29)	GAP-01 resuelto técnicamente (target_completion_at)
Fases 18 a 21	Rediseño, movimiento accesible, CSS 3D en la portada y QA visual	Hallazgos F21 A1–A4, V1–V3, F19-R y M1, corregidos
Fases 22 a 26	UML, PowerDesigner, Formato 09, QA global y release v1.1	F25-M01 y F25-M02 corregidos; F25-L03 abierto (formato)
F27 a F29H	Formatos académicos F2–F11, variables (F29C), plan, casos, QA final, métricas e informe	Sin fallos en la QA final

CAPÍTULO 9

CONTROL DE VERSIONES Y GESTIÓN DEL REPOSITORIO

Este capítulo describe cómo se gestionó el código fuente del proyecto mediante herramientas de control de versiones.

9.1. Repositorio del Proyecto

Repositorio Git publicado en GitHub (luivmz/saas-reclutamiento-multiempresa). Al 30/09/2026, develop tiene 145 commits (33 merges) y existen las etiquetas v1.0.0-

academic (9a946c2, línea base académica) y v1.1.0-academic. Los secretos (.env, .env.e2e) no se versionan.

9.2. Estrategia de Control de Versiones

- Git Flow simplificado: main estable, develop de integración y ramas feature/* por fase.
- Cada fase se integra con `git merge --no-ff` tras su auditoría y con la CI en verde.
- Commits convencionales (feat, test, docs, chore) y checkpoints TDD (RED) como commits propios.
- Nunca `push --force`, rebase de historia publicada ni etiquetas movidas.

9.3. Gestión de Ramas del Proyecto

Rama	Uso
main	Versiones publicadas (v1.0 y v1.1 académicas)
develop	Integración de todas las fases
feature/*	Una rama por fase (por ejemplo, feature/f29d-h-finalization)

9.4. Registro de Commits Relevantes

Commit	Descripción
e87ef39	Bootstrap de la aplicación Laravel
3e66623	Checkpoint RED de evaluaciones y entrevistas (TDD)
7a65bee	Ranking, decisión final humana, selección y cierre
a77c916	Suite E2E de Cypress completa
ccf05d1	Imágenes Docker fijadas y entorno E2E aislado
9a946c2	Etiqueta v1.0.0-academic
2b97fe3	Cierre de la Fase 25 (QA global v1.1)
bc44303	Integración de la F29C (variables); commit probado en la QA final

CAPÍTULO 10

DOCKERIZACIÓN Y DESPLIEGUE DE MÓDULOS

Este capítulo describe el proceso de dockerización del sistema y cómo se encapsulan los componentes del proyecto en contenedores.

10.1. Introducción a Docker en el Proyecto

Docker Compose permite levantar el sistema completo con un solo comando (`docker compose up -d --wait`) y garantiza que las pruebas y la demostración usen las mismas versiones. Todas las imágenes están fijadas.

10.2. Dockerización del Backend

Imagen `reclutamiento-php:dev` (PHP 8.4.25 con Composer y Node). El contenedor `app` instala dependencias si faltan, compila los assets, migra y sirve la aplicación; `queue` procesa las notificaciones. Su `healthcheck GET /health` verifica la base de datos y Redis.

10.3. Dockerización del Frontend

El frontend no tiene contenedor propio: Vite compila los assets dentro de la imagen de la aplicación (npm run build) y Laravel los sirve con Inertia. TypeScript, Vitest y el build se ejecutan en el mismo contenedor.

10.4. Orquestación con Docker Compose

Servicio	Imagen	Función
app	reclutamiento-php:dev	Aplicación (healthcheck /health)
queue	reclutamiento-php:dev	Worker de notificaciones
postgres	postgres:17.11-alpine	Bases de desarrollo, pruebas y E2E
redis	redis:7.4.11-alpine	Sesiones, caché y colas
app-e2e, queue-e2e	reclutamiento-php:dev	Entorno aislado para Cypress (perfil e2e)
cypress	cypress/including:15.3.0	Suite E2E

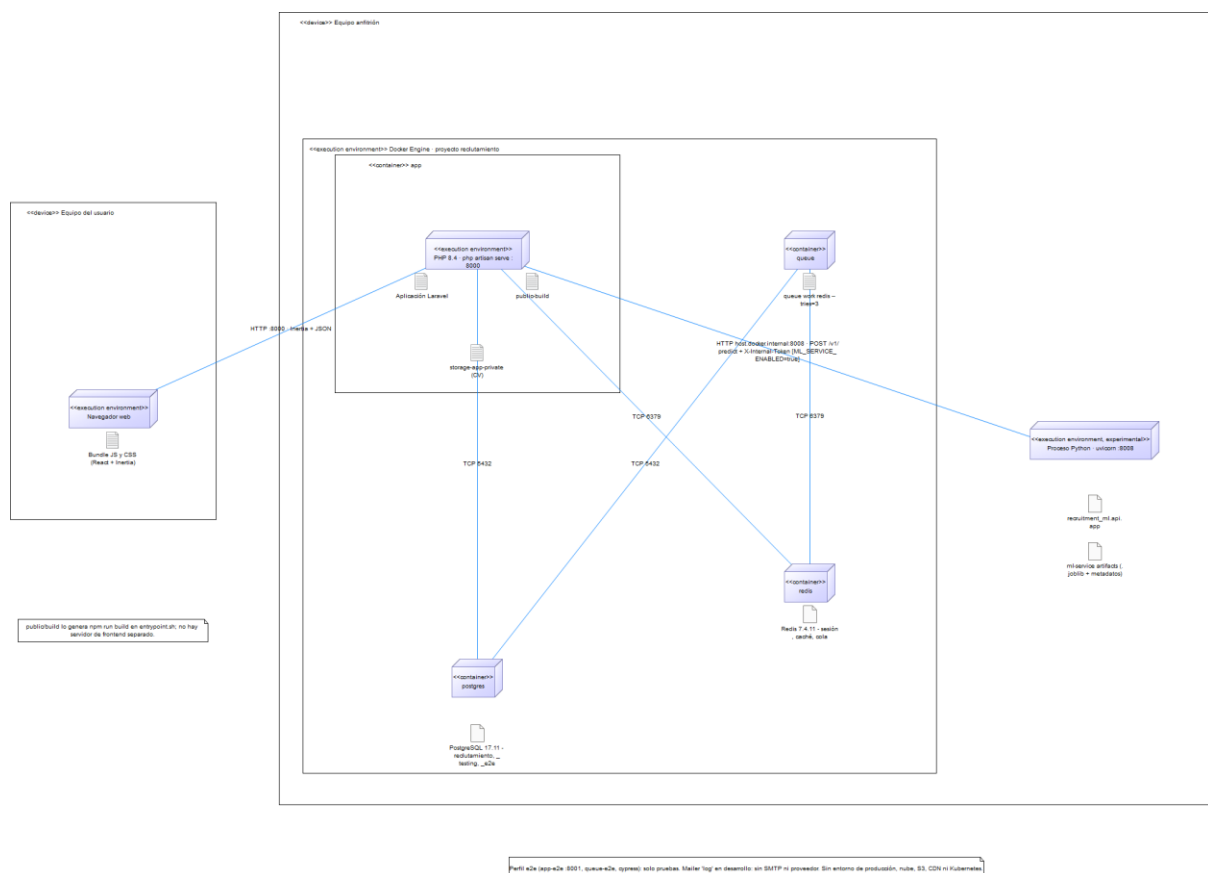


Figura 4. Diagrama de despliegue DE-01 (PowerDesigner, F23).

CAPÍTULO 11

ESTRATEGIA DE PRUEBAS DE SOFTWARE

Este capítulo describe la estrategia general de pruebas definida para garantizar la calidad del sistema desarrollado. El objetivo es verificar que el software cumple con los requerimientos funcionales y no funcionales definidos durante el análisis del proyecto.

11.1. Enfoque de Pruebas del Proyecto

Enfoque guiado por pruebas: cada regla se especifica primero como una prueba que falla y la automatización tiene prioridad sobre la prueba manual. La estrategia completa está en el Plan de Pruebas (F29D). Los datos son siempre ficticios y las bases de prueba están aisladas de la de demostración.

11.2. Niveles de Pruebas Aplicados

Nivel	Herramienta	Alcance
Unitario	PHPUnit (tests/Unit)	Estados, validadores y servicio de ranking
Integración / funcional	PHPUnit (tests/Feature) sobre PostgreSQL real	RF-01 a RF-27, autorización, notificaciones, auditoría
Sistema (E2E)	Cypress 15.3.0	Flujos por rol y flujo completo en navegador
Componentes	Vitest	Componentes de interfaz y accesibilidad
Aceptación	Flujo E2E-13 y recorrido manual (Fase 8)	Sin aceptación institucional

11.3. Tipos de Pruebas Ejecutadas

Distribución de los 128 casos de prueba de la F29E por tipo (el detalle está en docs/academico/casos-prueba/):

Tipo de prueba	Casos
Aceptación (flujo completo)	1
Aceptación (manual)	2
Estática / build	3
Integración / funcional	42
ML experimental	28
Multitenencia	4
No funcional	2
Regresión	4
Regresión / CI	1
Seguridad básica	15
UI / E2E	10
UI / E2E (accesibilidad)	4
UI / componentes	6
Unitaria	6

11.4. Plan de Ejecución de Pruebas

Orden de ejecución de cierre (F29F), con el árbol de trabajo limpio y la salida completa de cada herramienta guardada como evidencia: configuración de Compose → PHPUnit → TypeScript → build → Vitest → pytest → Cypress. Cualquier fallo se clasifica (regresión real, flaky, ambiente, dependencia externa, documental o no aplicable) y nunca se oculta; el runtime no se corrige sin autorización del equipo.

CAPÍTULO 12

AUTOMATIZACIÓN DE PRUEBAS

Este capítulo describe el proceso de automatización de pruebas del sistema. La automatización permite ejecutar pruebas de manera repetible y eficiente, reduciendo errores humanos y mejorando el control de calidad.

12.1. Herramientas de Automatización

Herramienta	Suite	Archivos
PHPUnit 12	Unitarias e integración	6 unitarios, 42 de integración
Cypress 15.3.0	E2E	20 specs
Vitest (vp test)	Componentes	6 archivos
pytest	Servicio ML experimental	18 archivos
GitHub Actions	Integración continua	.github/workflows/tests.yml

12.2. Configuración del Entorno de Pruebas

- PHPUnit fuerza DB_DATABASE=reclutamiento_testing en phpunit.xml (DEF-02) y usa RefreshDatabase.
- Cypress corre en su imagen oficial contra app-e2e, con .env.e2e generado por npm run e2e:setup (token y clave aleatorios, nunca versionados) y reset por endpoint protegido.
- Sin reintentos en Cypress (retries: 0): un fallo intermitente se investiga.
- El servicio ML se prueba en su entorno virtual con dataset sintético y contrato congelado.

12.3. Scripts de Pruebas Automatizadas

Las pruebas viven con el código: tests/Unit, tests/Feature, cypress/e2e, resources/js/**/*test.tsx y ml-service/tests. Los nombres test_rfNN_* enlazan cada prueba con su RF, y la matriz de trazabilidad (F29E) enlaza RF → CU → caso de prueba → prueba automatizada → evidencia.

12.4. Ejecución Automática de Pruebas

GitHub Actions ejecuta en cada push a develop y main la instalación de dependencias, el build, TypeScript y PHPUnit. Cypress, Vitest y pytest se ejecutan en local con Docker y el entorno virtual.

Rama	Commit	Resultado
develop	bc44303	success
main	60ebcb2	sin ejecución: pendiente de ejecución manual

CAPÍTULO 13

MÉTRICAS DE CALIDAD

Este capítulo presenta los resultados obtenidos durante la ejecución de las pruebas del sistema. Además, se analizan métricas de calidad basadas en estándares internacionales.

13.1. Ejecución de Casos de Pruebas

Ejecución QA final del 30/09/2026 sobre el commit bc44303 (F29F). Resultados reales:

Suite	Ejecutadas	Aprobadas	Fallidas	Omitidas
PHPUnit	419	411	0	8
Cypress	85	85	0	0
Vitest	42	42	0	0

Suite	Ejecutadas	Aprobadas	Fallidas	Omitidas
pytest	533	533	0	0

TypeScript sin errores y build sin errores. Las 8 omitidas de PHPUnit dependen de la verificación de correo, desactivada en Fortify. Se cumplen los siete criterios de aceptación del plan.

13.2. Registro de Defectos

Registro unificado de la F29G: 31 registros (13 de la v1.0 y 18 de la v1.1, incluidos hallazgos documentales y de proceso). Por severidad: 2 críticos, 6 altos, 14 medios y 9 bajos. Cerrados: 29. La QA final no produjo defectos nuevos del software.

Abierto	Severidad	Estado
F25-L03	Baja	Abierto (remedido en la F29F: OBS-F29F-05)
OBS-F29F-04	Baja	Abierto: CI main F29C pendiente de ejecución manual

13.3. Métricas de Calidad del Software

Métrica	Valor
Tasa de aprobación (aprobadas / ejecutadas no omitidas)	100 % en PHPUnit, Cypress, Vitest y pytest
Cobertura funcional de RF (CP automatizado aprobado)	27 de 27
CU cubiertos a través de sus RF	20 de 20
Casos automatizados	124 de 128
Estabilidad	Resultados idénticos a la QA de release F25 y entre las dos ejecuciones de PHPUnit
Cobertura de código	NO MEDIDA (sin Xdebug ni PCOV)
Rendimiento y disponibilidad	NO MEDIDOS (RNF-06 y RNF-07 no verificados)

13.4. Evaluación de Calidad basada en Estándares

Evaluación organizada con ISO/IEC 25010, sin certificación: seguridad (confidencialidad, integridad y responsabilidad) verificada por RNF-01 a RNF-04 y RNF-10; usabilidad, compatibilidad y mantenibilidad con evidencia parcial; eficiencia de desempeño y fiabilidad no verificadas. Detalle en docs/academico/metricas-calidad/.

CAPÍTULO 14

IMPLEMENTACIÓN Y MONITOREO

Este capítulo describe la fase final de implantación del sistema y las actividades de monitoreo realizadas después de su implementación. El objetivo es asegurar que el sistema funciona correctamente en su entorno de ejecución.

14.1. Preparación del Entorno de Implementación

La implementación es un **entorno de desarrollo y demostración** con Docker: no existe entorno de producción. Requisitos: Docker Desktop, Git y el .env generado desde .env.example (docs/docker.md).

14.2. Implementación del Sistema

Instalación: clonar el repositorio, `docker compose up -d --wait` y cargar los datos ficticios con `docker compose exec app php artisan migrate:fresh --seed`. La aplicación queda en `http://localhost:8000` con usuarios demo (docs/demo-users.md). El servicio ML es opcional y se activa con `ML_SERVICE_ENABLED=true`.

14.3. Verificación de Funcionamiento

La verificación de funcionamiento es la ejecución QA final (F29F): servicios healthy, suites sin fallos y flujo completo E2E-13 aprobado. No se verificó en un entorno institucional ni con usuarios reales.

14.4. Monitoreo del Sistema

Mecanismo	Uso
GET /health	Estado de la base de datos y Redis
Healthchecks de Compose	app, queue, postgres y redis
Logs de Laravel	storage/logs y salida del contenedor
Tabla failed_jobs	Trabajos de cola fallidos
Auditoría	audit_logs de solo inserción, consultable por Dirección

No hay monitoreo de errores en producción, alertas ni SLA: no se afirma disponibilidad ni rendimiento verificados.

CONCLUSIONES

- Se implementó y verificó una plataforma SaaS multiempresa que cubre los 27 RF de la línea base, con la decisión final de selección siempre humana (RF-23).
- La QA final del 30/09/2026 no registró fallos en ninguna suite y cumplió los siete criterios de aceptación; los 27 RF tienen al menos un caso automatizado aprobado.
- El proceso de pruebas (TDD, regresión automática, CI y registro de defectos con prueba de regresión) dejó trazabilidad completa RF → CU → caso de prueba → prueba → evidencia.
- El componente de riesgo operacional (RF-29) quedó como EXPERIMENTAL: validado solo con datos sintéticos, no evalúa personas y no está autorizado para producción.
- Los beneficios sobre el proceso del colegio no se midieron: la relación entre la plataforma y la gestión del proceso es una hipótesis (TO-BE PROPUESTO) y el AS-IS sigue preliminar.

RECOMENDACIONES

Trabajo futuro (fases posteriores; nada de esto está implementado):

- Validar el AS-IS y el TO-BE con RR. HH. y la Administración del colegio.
- Ejecutar la CI de main pendiente (OBS-F29F-04) y aplicar el formato pendiente en un commit propio (F25-L03).
- Medir cobertura de código (PCOV) y agregar análisis estático.
- Pruebas de carga y objetivos de rendimiento (RNF-06); estrategia de disponibilidad y respaldo (RNF-07).
- Pruebas dinámicas de seguridad (OWASP ZAP) y Row Level Security en PostgreSQL.

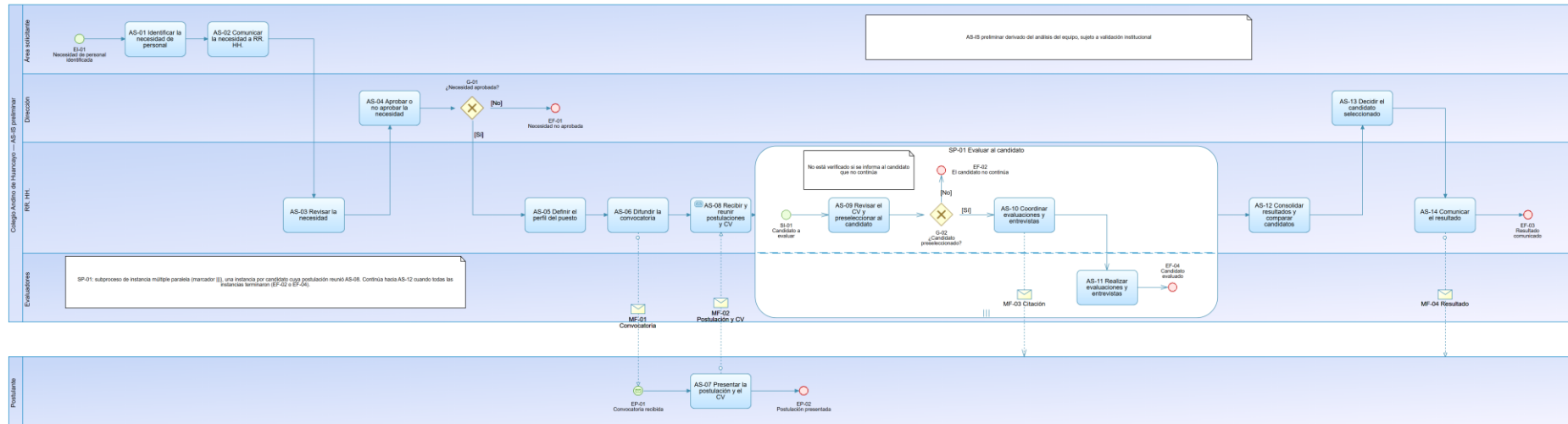
- Monitoreo de errores y alertas; despliegue con imagen de producción y HTTPS.
- Decidir la promoción de RF-28 y RF-29 (candidatos) y validar institucionalmente cualquier uso del ML.

REFERENCIAS

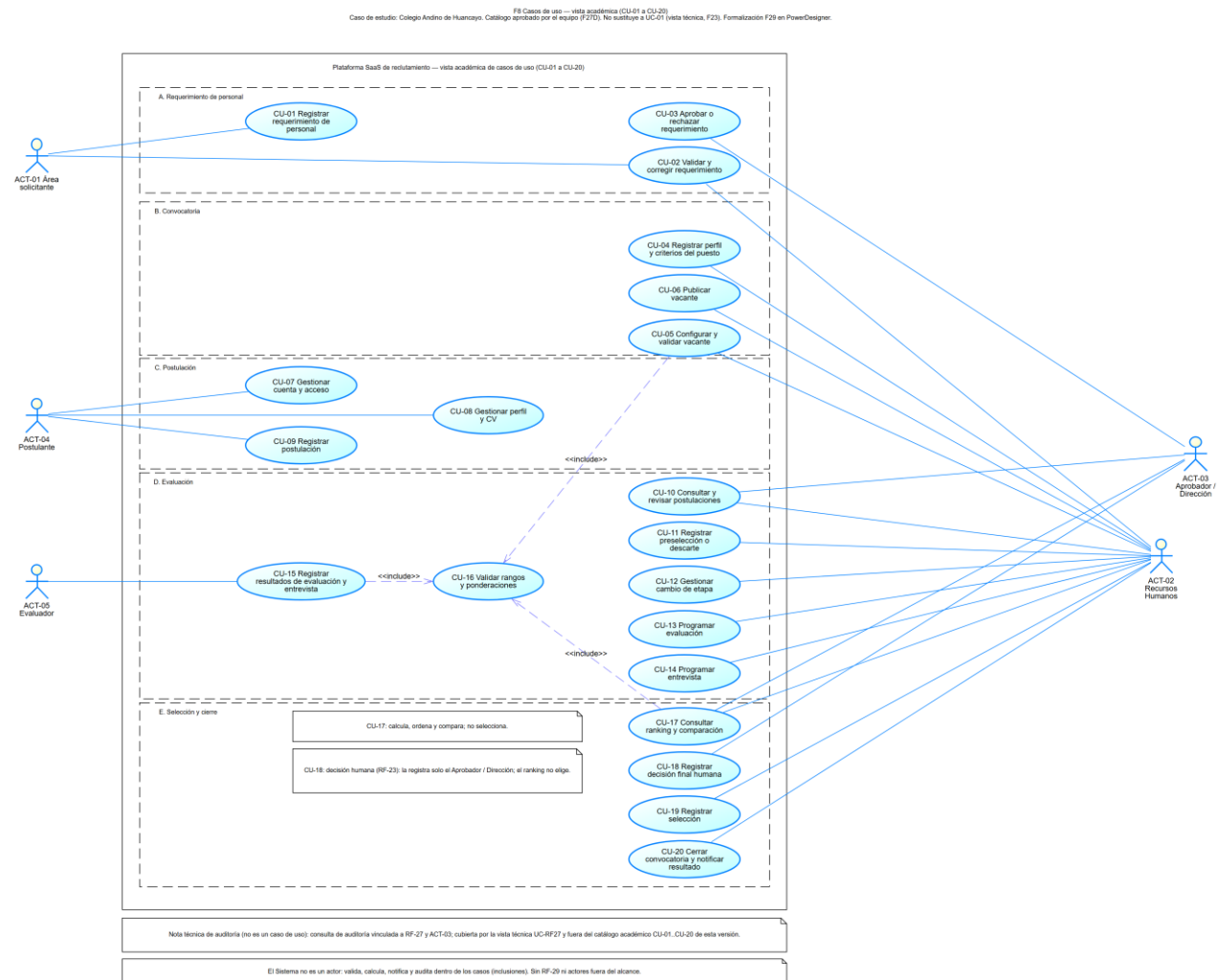
- Universidad Continental. Guías de práctica 02 a 11 y Formatos oficiales del curso Pruebas y Calidad de Software (docs/academico/00-fuentes-oficiales/).
- Guía de laboratorio E1: Desarrollo de Software con Inteligencia Artificial (docs/academico/00-fuentes-oficiales/guias-ia/).
- Plantilla de Plan de Pruebas de Software del curso y Plantilla de estructura del proyecto final.
- ISO/IEC 25010, Systems and software quality models (usada como marco de referencia).
- W3C. Web Content Accessibility Guidelines (WCAG) 2.2.
- Documentación oficial de Laravel, React, Inertia, PostgreSQL, Redis, Docker, PHPUnit, Cypress, Vitest, pytest, scikit-learn y FastAPI.
- Repositorio del proyecto: documentación técnica (docs/), trazabilidad (docs/final-report/traceability-master.md) y entregables académicos (docs/academico/).

ANEXOS

Anexo	Contenido	Ubicación
A	BPMN AS-IS (PowerDesigner)	Página siguiente; docs/academico/practica-03
B	BPMN TO-BE (PowerDesigner)	Página siguiente; docs/academico/practica-05
C	Diagrama de casos de uso (PowerDesigner)	Página siguiente; docs/academico/practica-08
D	Formatos académicos F2 a F11 (incluido F11-R) y alcance F9	docs/academico/practica-02 a practica-11; phase-24/output
E	Variables y matriz de operacionalización (F29C)	docs/academico/operacionalizacion/
F	Plan de Pruebas (F29D)	docs/academico/plan-pruebas/
G	Casos de prueba y matriz de trazabilidad (F29E)	docs/academico/casos-prueba/
H	Ejecución QA final y evidencias (F29F)	docs/academico/qa-final/
I	Defectos y métricas (F29G)	docs/academico/metricas-calidad/
J	Modelos PowerDesigner y exportaciones	docs/academico/powerdesigner/; docs/v1.1/powerdesigner/
K	Docker, TDD y trazabilidad técnica	docs/docker.md; docs/tdd-evidence.md; docs/final-report/



Anexo A. BPMN AS-IS del proceso de reclutamiento (AS-IS PRELIMINAR; PowerDesigner, F29).



Anexo C. Diagrama de casos de uso CU-01 a CU-20 (PowerDesigner, F29).